Neural Language Models: From Fixed Windows to Recurrent Networks

CS224N: Natural Language Processing with Deep Learning
Stanford University

Lecture 5 Study Notes

Contents

1 Introduction to Neural Language Models

1.1 Overview

Neural language models use neural networks to estimate probability distributions over sequences of words. The fundamental task remains: given a sequence of words $w_1, w_2, \ldots, w_{t-1}$, predict the probability distribution over the next word w_t.

1.2 Key Concept

Previously, we used concatenated word vectors as context to make predictions (e.g., location vs. non-location classification). We can extend this idea to predict the next word in a sequence, creating a neural language model.

Core Idea

Instead of binary or simple classification, use neural networks to perform multi-class classification over the entire vocabulary to predict the next word.

2 Fixed Window Neural Language Models

2.1 Architecture

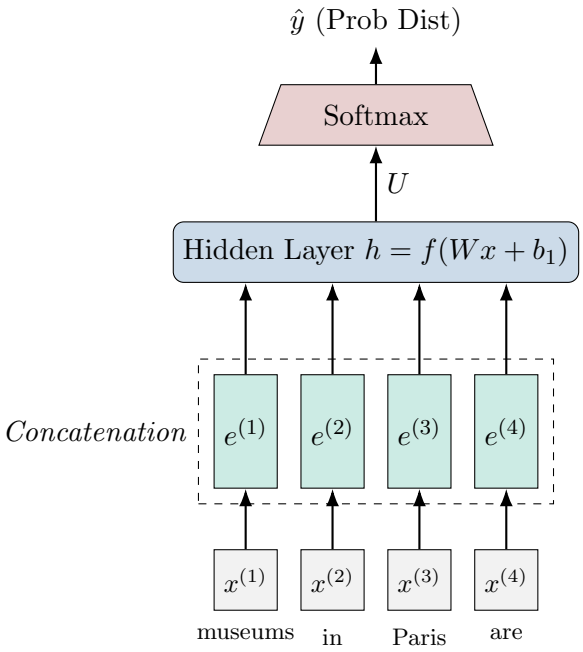


Figure 1: A Fixed-Window Neural Language Model processing 4 words.

2.2 Mathematical Formulation

Input: Fixed window of n previous words: $w_{t-n}, \ldots, w_{t-1}$

Step 1 - Embedding Lookup:

$$e_i = E \cdot w_i \quad \text{where } w_i \text{ is one-hot vector}$$

Step 2 - Concatenation:

$$x = [e_{t-n}; e_{t-n+1}; \ldots; e_{t-1}]$$

Step 3 - Hidden Layer:

$$h = \tanh(W \cdot x + b_1)$$

Step 4 - Output Layer:

$$\hat{y} = \text{softmax}(U \cdot h + b_2)$$

Step 5 - Prediction:

$$P(w_t | w_{t-n}, \ldots, w_{t-1}) = \hat{y}_{w_t}$$

2.3 Historical Context: Bengio et al. (2003)

- **Proposer:** Yoshua Bengio and colleagues, early 2000s
- **Innovation:** First neural language model to replace n-gram models
- **Initial reception:** Limited adoption due to:
  - Computational constraints (pre-GPU era)
  - Fixed window limitation (still uses Markov assumption)
  - N-gram models scaled better with more data at the time

2.4 Advantages of Fixed Window Neural LM

1. **No sparsity problem:** Similar contexts produce similar predictions due to learned embeddings
2. **Compact storage:** Only need to store network parameters, not all observed n-grams
3. **Better generalization:** Can handle unseen word combinations through distributed representations

2.5 Limitations of Fixed Window Neural LM

1. **Markov assumption:** Still limited to fixed context window
2. **No fixed window is large enough:** Cannot capture arbitrarily long dependencies
3. **Position-specific parameters:** Different weight matrix columns for different positions
   - Word "student" at position 1 uses different parameters than "student" at position 3
   - Inefficient: should recognize "student" indicates likely words (books, exams, homework) regardless of position
4. **Enlarging window:** Increases parameter count dramatically (W matrix grows)

Key Problem

We need an architecture that:

- Processes any length input
- Uses the same parameters for words regardless of position
- Maintains efficiency as context grows

This motivates **Recurrent Neural Networks**.

3 Recurrent Neural Networks (RNNs)

3.1 Neural Network Architecture Progression

In this course, we encounter several neural architectures:

1. **Word2Vec:** Simple encoder-decoder architecture
2. **Feed-forward Networks:** Fully connected layers (classic neural networks)
3. **Recurrent Neural Networks (RNNs):** Sequential processing with memory
4. **Transformers:** (covered later) Attention-based architectures

3.2 Core Concept

Idea: Apply the same set of weights at successive time steps, updating a hidden state as we process the sequence.

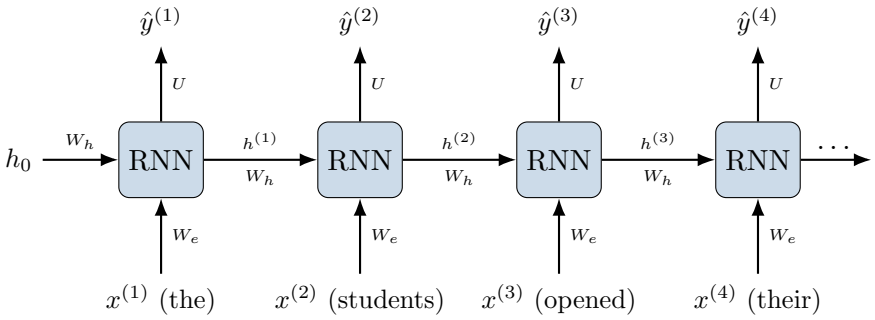


Figure 2: Unrolled Recurrent Neural Network. Note that W_h, W_e, U are shared.

3.3 RNN Computation

Initialization:

$$h_0 = \vec{0} \quad \text{(or learned initialization)}$$

At each time step t:

$$h_t = \tanh(W_h \cdot h_{t-1} + W_e \cdot e_t + b) \tag{1}$$

$$\text{where } e_t = E \cdot x_t \quad \text{(word embedding)} \tag{2}$$

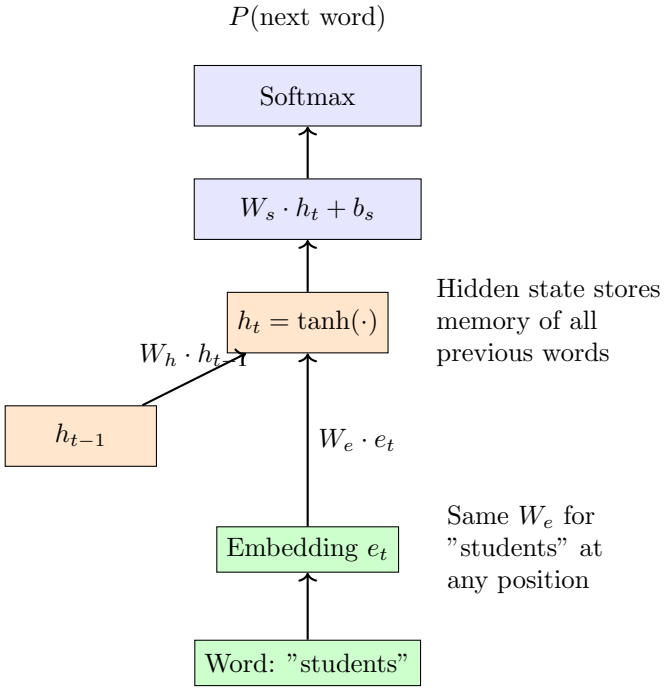
Output (prediction):

$$\hat{y}_t = \text{softmax}(W_s \cdot h_t + b_s)$$

Key Components

- W_h: Hidden-to-hidden weight matrix (recurrent weights)
- W_e: Embedding-to-hidden weight matrix (input weights)
- W_s: Hidden-to-output weight matrix
- h_t: Hidden state at time t (memory of everything seen so far)
- tanh: Non-linearity (balanced on positive/negative)

3.4 Detailed RNN Architecture



3.5 Advantages of RNNs

1. **Any-length sequences:** Can process arbitrarily long input sequences
2. **No size increase:** Model size doesn't grow with sequence length
   - Computation increases, but parameters remain constant
   - Hidden state h has fixed dimension
3. **Symmetry:** Same weights applied at every time step
4. **Position-invariant:** Word "student" processed with same weights regardless of position
5. **Theoretically unlimited memory:** Hidden state can encode entire history

3.6 Disadvantages of RNNs

1. **Sequential computation is slow**
   - Cannot parallelize across time steps
   - Must compute h_1 before h_2 before h_3, etc.
   - Requires a for-loop: `for t = 1 to T`
   - Contrary to efficient vectorized computation
2. **Vanishing gradient / long-term dependencies**
   - In theory: hidden state encodes entire history
   - In practice: recent words dominate the hidden state
   - Information from distant past fades quickly
   - Simple RNNs forget too quickly

Important Note

The sequential computation problem is a major reason RNNs have fallen out of favor, leading to the development of Transformers which can parallelize computation.

4 Training RNN Language Models

4.1 Training Process Overview

1. Obtain large corpus of text
2. For each time step, compute prediction of next word
3. Compare prediction to actual next word
4. Compute loss (cross-entropy)
5. Update parameters via backpropagation

4.2 Loss Function

Cross-Entropy Loss at time t:

$$J^{(t)}(\theta) = -\log P(w_t|w_1, \ldots, w_{t-1})$$

Overall objective (average loss):

$$J(\theta) = \frac{1}{T}\sum_{t=1}^{T} J^{(t)}(\theta) = -\frac{1}{T}\sum_{t=1}^{T} \log P(w_t|w_1, \ldots, w_{t-1})$$

Ideal scenario:

- Predict actual next word with probability 1
- $-\log(1) = 0$ (no loss)

Realistic scenario:

- Predict actual next word with probability $p < 1$
- Loss $= -\log(p)$
- Lower probability $\rightarrow$ higher loss

4.3 Training Example: Teacher Forcing

4.4 Teacher Forcing Explained

Process:

1. Given prefix (e.g., "the"), predict next word distribution
2. Calculate loss based on actual next word ("students")
3. **Feed actual word** "students" as next input (not predicted word)
4. Repeat for next prediction

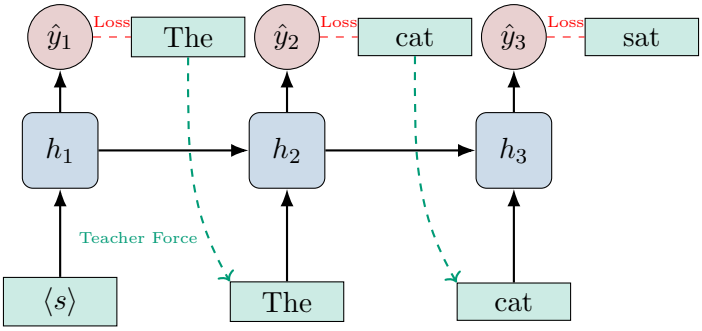


Figure 3: Teacher Forcing: The input at $t+1$ is the *actual* label from t, not the prediction.

Why Teacher Forcing?

- Keeps model aligned with real text
- Prevents error accumulation
- Simpler training dynamics
- Faster convergence

Limitation:

- Model doesn't explore its own generations during training
- May behave differently during free generation
- Exposure bias problem

4.5 Practical Training Considerations

4.5.1 Segmentation

Problem: Cannot run RNN over billions of words in one sequence

- Memory constraints
- Gradient computation issues

Solution: Divide corpus into segments
Options:

1. Linguistically motivated: sentences or documents
2. Practical approach: Fixed-length chunks (e.g., 100 words)
    - Easier to batch (all segments same length)
    - More efficient GPU utilization
    - Better vectorization

Practical Note

Modern implementations typically use fixed-length segments (e.g., 100-200 tokens) for efficiency, even though this reintroduces a form of Markov assumption.

4.5.2 Backpropagation Through Time (BPTT)

Computing Gradients for Repeated Weights:
The weight matrix W_h appears at every time step. How do we compute $\frac{\partial J}{\partial W_h}$?
Solution:

$$\frac{\partial J}{\partial W_h} = \sum_{t=1}^{T} \frac{\partial J^{(t)}}{\partial W_h}$$

Intuition:

- Pretend W_h at each position is different: $W_h^{(1)}, W_h^{(2)}, \ldots$
- Compute gradients for each
- Sum them (gradient at outward branches)
- Works because copies are identity operations

4.5.3 Truncated Backpropagation Through Time

Motivation: Full BPTT over 100 time steps is expensive
Truncated BPTT:

- Forward pass: Use full sequence (all 100 steps)
- Backward pass: Only backpropagate for k steps (e.g., $k = 20$)
- Speeds up training
- Often sufficient in practice

5 Text Generation with RNN Language Models

5.1 Generation Process

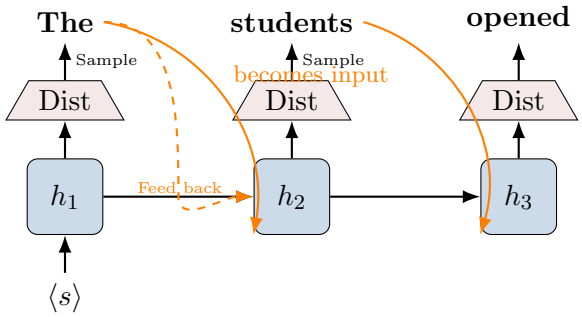


Figure 4: Inference: The sampled output becomes the next input.

5.2 Generation Algorithm

1. **Initialize:** $h_0 = \vec{0}$, input $= \langle s \rangle$ (start token)
2. **Forward pass:** Compute h_1 and output distribution $\hat{y}_1$
3. **Sample:** Draw word $w_1 \sim \hat{y}_1$

4. **Feed back:** Use w_1 as input for next step
5. **Repeat:** Continue until $\langle/s\rangle$ (end token) is generated

Special Tokens:

- $\langle s\rangle$: Start of sequence
- $\langle/s\rangle$: End of sequence (stopping criterion)

Probabilistic Nature:

- Sampling introduces randomness
- Multiple runs produce different outputs
- Same mechanism used in ChatGPT and other LLMs

5.3 Generation Examples

5.3.1 Example 1: Barack Obama Speeches

Training corpus: Few hundred thousand words

Generated text:

> *"The United States will step up to the cost of a new challenges of the American people that will share the fact that we created the problem they were attacked and so that they have to say that all the task of the final days of war that I will not be able to get this done"*

Observation: Somewhat coherent, political vocabulary, but lacking long-term coherence

5.3.2 Example 2: Harry Potter

Training corpus: Complete Harry Potter series

Generated text:

> *"Sorry Harry shouted panicking I'll leave those brooms in London are they no idea said nearly headless Nick casting low close by Cedric carrying the last bit of trial charms from Harry's shoulder and to answer him the common room perched upon it forearms held a shining knob from when the spider hadn't felt it seamed he reached the teams too well there you are"*

Observation: Character names correct, magical vocabulary, but plot/logic inconsistent

5.3.3 Example 3: Recipe Generation

Generated recipe: "Chocolate Ranch Barbecue"

> Categories: game casseroles cookies cookies
> Yield: six servings
> 2 tablespoons of Parmesan cheese chopped
> 1 cup of coconut milk and three eggs beaten
> Place each pasture over layers of lumps
> Shape mixture into the moderate oven and simmer until firm
> Serve hot and bodied fresh mustard orange and cheese
> Combine the cheese and salt together the dough in a large Skillet

Observation: Recipe-like structure and vocabulary, but semantically nonsensical

6 Character-Level RNNs

6.1 Concept

Instead of word-level tokens, use character-level tokens:

- Each time step processes one character
- Vocabulary size: 100 (letters, digits, punctuation)
- Can generate novel "words"

6.2 Applications

6.2.1 Bioinformatics

- DNA/RNA sequences
- Protein sequences
- Gene prediction

6.2.2 Paint Color Name Generation

Task: Given RGB values, generate paint color name

Architecture:

- Initialize h_0 with RGB values (instead of zeros)
- Generate characters sequentially
- Model learns correlation between color and name

Examples:

- Gasty Pink
- Power Gray
- Naval Tan
- Bank Co White
- Stoner Blue
- Dope (for a specific brown color)
- Stinky Bean
- Turley

Observation: Surprisingly creative and plausible-sounding names!

7 Summary and Key Takeaways

7.1 Fixed Window Neural LM

- First neural approach to language modeling (Bengio et al., 2003)
- Advantages: No sparsity, better generalization
- Limitations: Fixed context, position-specific parameters

7.2 Recurrent Neural Networks

- Process sequences of any length
- Maintain hidden state as memory
- Same weights at all positions
- Disadvantages: Sequential computation, vanishing gradients

7.3 Training

- Cross-entropy loss with teacher forcing
- Backpropagation through time (BPTT)
- Truncated BPTT for efficiency
- Segmentation for practical training

7.4 Generation

- Rollout process with sampling
- Special start/end tokens
- Probabilistic output (different runs → different results)
- Works at word or character level

Looking Forward

RNNs were revolutionary but have limitations:

- Sequential computation (slow)
- Vanishing gradient problem
- Difficulty with long-range dependencies

Next topics will cover:

- LSTM and GRU (addressing vanishing gradients)
- Attention mechanisms
- Transformers (parallel computation)